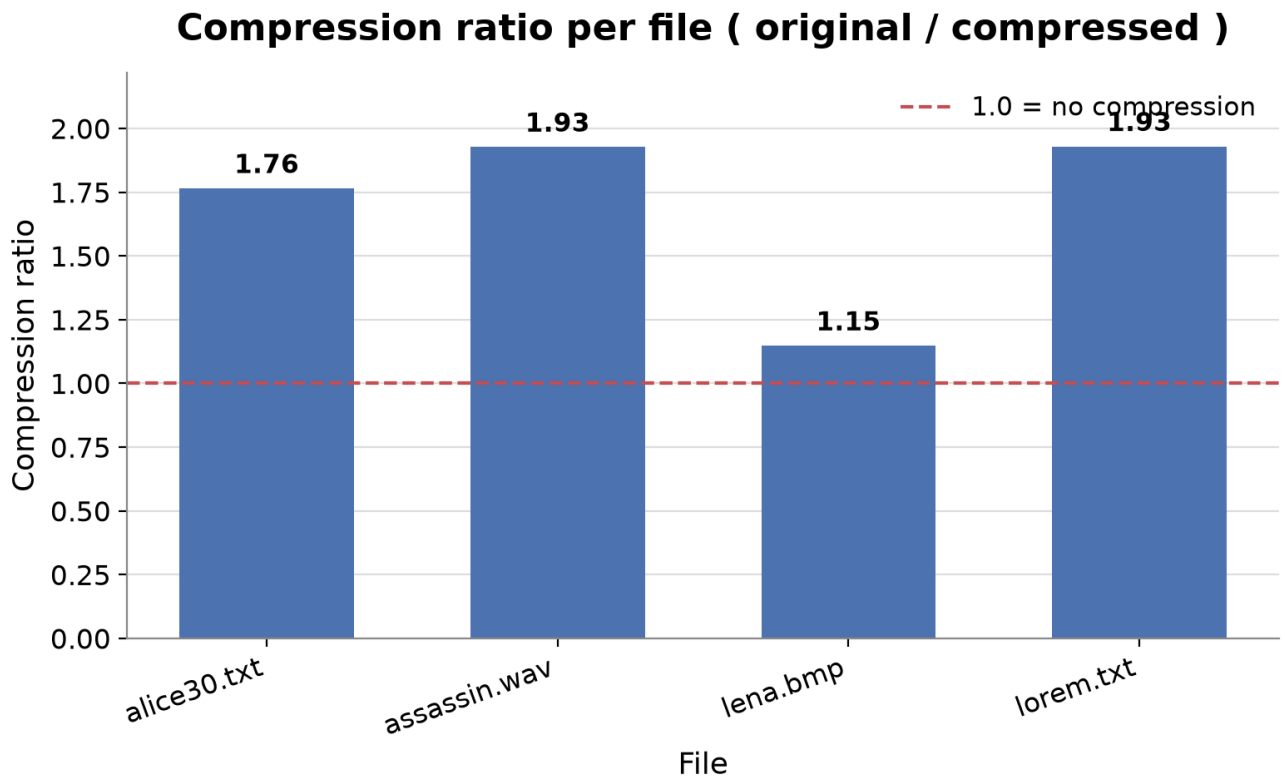
Poročilo – Vaja 1

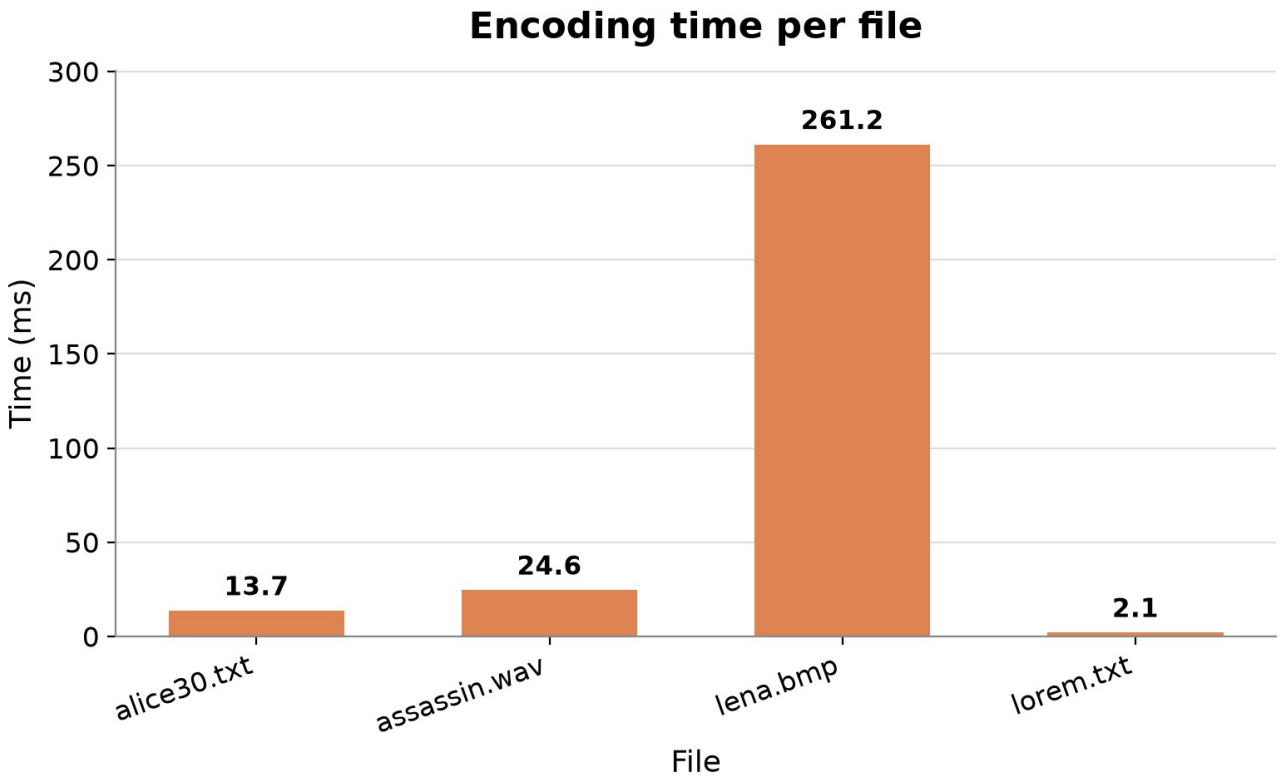
Andrija Obradović

1. Graf kompresijskega razmerja



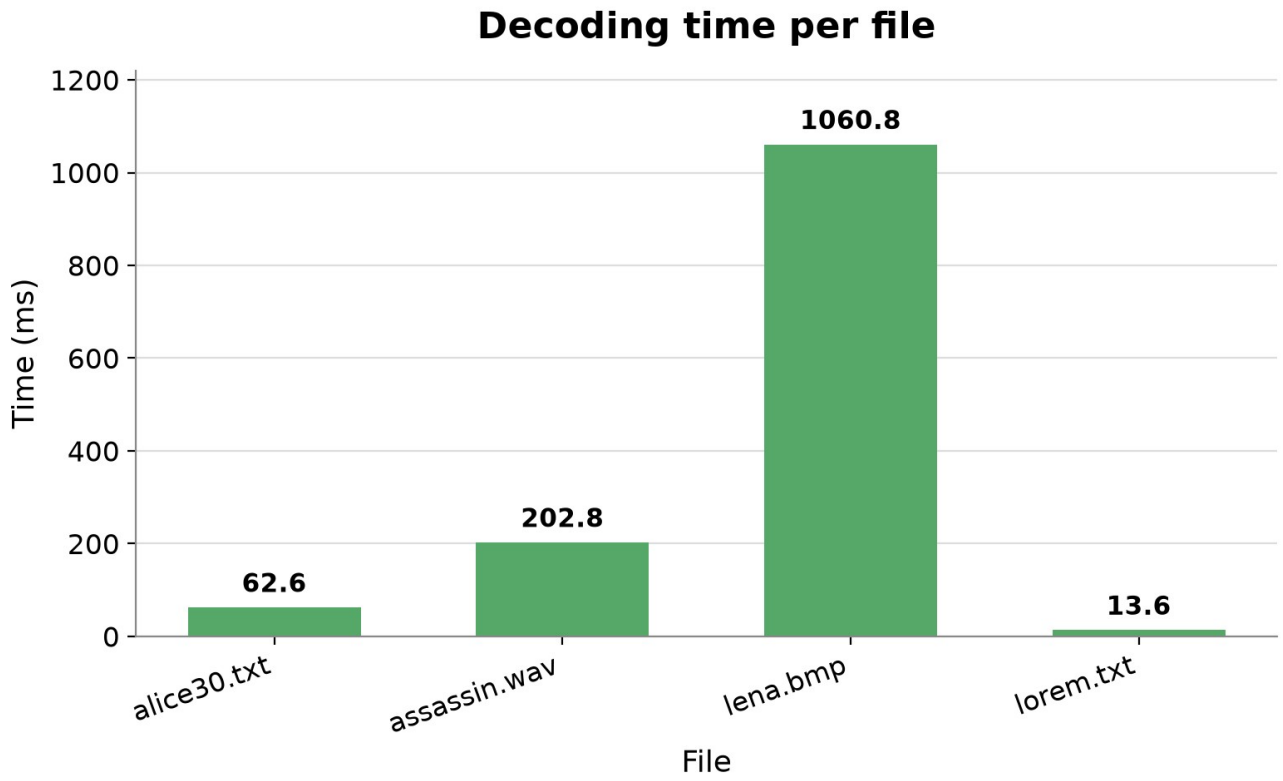
Graf 4: Kompresijsko razmerje je izračunano kot velikost originalne datoteke deljeno z velikost stisnjene datoteke. Višja vrednost pomeni boljšo kompresijo. Črka pri vrednosti 1 označuje primer brez kompresije.

2. Graf časa kompresije



Graf 8: Povprečni čas kompresije za posamezne testne datoteke v milisekundah, izmerjen na 100 ponovitev.

3. Graf časa dekompresije



Graf 9: Povprečni čas dekompesije za posamezne testne datoteke v milisekundah, izmerjen na 100 ponovitev.

4. Kako sem kodiral tabelo verjetnosti

```cpp
//Frequency table
std::array<Symbol, 256> table{};
uint32_t cumulativeFrequency = 0;

char byte;
while (input.get(byte)) {
    ++table[static_cast<unsigned char>(byte)].frequency;
}

for (auto &symbol : table) {
    symbol.low = cumulativeFrequency;
    cumulativeFrequency += symbol.frequency;
    symbol.high = cumulativeFrequency;
}
```

```cpp
struct Symbol {
    uint32_t frequency = 0;
    uint32_t low = 0;
    uint32_t high = 0;
};
```

Frekvenčno tabelo sem kodiral z poljem 256 elementov tipa Symbol, kjer vsak element predstavlja en možen bajt in vsebuje njegovo frekvenco, spodnjo in zgornjo mejo intervala. V prvem prehodu skozi datoteko preštejem pojavitev posameznih bajtov, torej zračunam frekvenc. Potem, v prehodu skozi posamezne simbole v tabeli, za vsak simbol zračunam še spodnjo in zgornjo mejo in hkrati zračunam kumulativnu frekvencu.

```cpp
//Header

//Number of bits (char -> 1 byte)
output.put(static_cast<char>(BITS));

//Symbol count so decoder knows when to stop reading symbols (uint16 -> 2 bytes)
uint16_t symbolCount = 0;
for (const auto& symbol : table) {
    if (symbol.frequency > 0) {
        symbolCount++;
    }
}

output.write(reinterpret_cast<const char*>(&symbolCount), sizeof(symbolCount));

//Symbols (symbol count bytes , 1 symbol -> 1 byte)
for (int i = 0; i < 256; i++) {
    if (table[i].frequency > 0) {
        output.put(static_cast<char>(i));
    }
}

//Frequencies (frequency writes 4 bytes for each symbol, 1 frequency -> 4 bytes)
for (int i = 0; i < 256; i++) {
    if (table[i].frequency > 0) {
        output.write(reinterpret_cast<const char*>(&table[i].frequency), sizeof(uint32_t));
    }
}
```

V glavo stisnjene datoteke zapišem število bitov kodirnika, število unikatnih simbolov, potem simbole od začetka do konca tabele in na koncu frekvence simbolov, po enakem vrstnem redu.

Na podlagi te glave dekoder dobi tabelo frekvenc pa si meje posamezneg simbola zračuna po podobnem načinu kot koder.

5. Kako bi lahko optimiziral algoritem

Prva mogoča optimizacija algoritma je v funkciji compress, kje dvakrat beremo vsebino datoteke, prvič za izračun verjetnoste tabele in potem za kodiranje simbolov. Lahko ob prvem branju vsebino datoteke shranimo v pominilnik, ex. Vector, in se izognemo ponovnega branja datotek.

Druga optimizacija je v funkciji decompress, kje linearno iščemo območje frekvenc simbolov. Lahko tako iskanje nadomestimo z binarnim iskanjem ali lookup tabelom.

Tretja optimizacija je pisanje in branje bitov v ali iz izhodne datoteke. V BitWriter in BitReader v tem času imam buffer ki je velikosti 8, torej počaka na 8 bitov in piše en bajt v izhodno datoteko. Lahko pa imamo večji buffer bajtov in tako zmanjšamo število IO operacij.